```
/**
 * Session-aware Logging Utility
 *
 * Provides a standardized way to log DML, errors, and success states with
 * ANSI color support and environmental filtering.
 *
 * @module LogUtil
 * @file log.util.ts
 *
 * Acknowledgments: Architecture for session-reactive logging and atomic DML utility
 * developed in collaboration with AI on Google Search, powered by the Gemini family of models.
 *
 * @copyright 2018-2026, Dennis Jorgenson
 */

import type { StatusCode } from "#app/lexicon";
import { SC, StatusMessage } from "#app/lexicon";
import { Session, type ISession } from "#app/session";
import { ApiError } from "#db/types";

// --- Color Configuration ---
const SYSTEM_NO_COLOR = process.env.NO_COLOR === "true";

// --- Killbox labels ---
const Red = (l: string) => (SYSTEM_NO_COLOR ? `[${l}]` : `\x1b[31m[${l}]\x1b[0m`);
const Green = (l: string) => (SYSTEM_NO_COLOR ? `[${l}]` : `\x1b[32m[${l}]\x1b[0m`);
const Yellow = (l: string) => (SYSTEM_NO_COLOR ? `[${l}]` : `\x1b[33m[${l}]\x1b[0m`);
const Blue = (l: string) => (SYSTEM_NO_COLOR ? `[${l}]` : `\x1b[34m[${l}]\x1b[0m`);
const Magenta = (l: string) => (SYSTEM_NO_COLOR ? `[${l}]` : `\x1b[35m[${l}]\x1b[0m`);

// --- Context label only ---
const Cyan = (l: string) => (SYSTEM_NO_COLOR ? `${l}` : `\x1b[36m${l}\x1b[0m`);

// --- Types & Interfaces ---

/**
 * @interface ILogFlags
 * @description Maps to APP_LOGGING JSON in .env to toggle specific log categories.
 */
interface ILogFlags {
  select: boolean;
  update: boolean;
  insert: boolean;
  delete: boolean;
  account: boolean;
  errors: boolean;
  success: boolean;
  [key: string]: boolean;
}

/** @description Shared signature for all Database Manipulation Language (DML) logs. */
type DmlLogger = (context: string, sql: string, cols?: string[], args?: any[], ...meta: any[]) => void;

const DML_METHODS = ["select", "update", "insert", "delete"] as const;

// 1. Define the interface based on the array above
type IDmlMethods = {
  [K in (typeof DML_METHODS)[number]]: DmlLogger;
};

/**
 * @interface ILogger
 * @description The logging instance passed into the withLog callback.
 */
export type ILogger = {
  success: (context: string, message?: string, code?: StatusCode) => void;
  error: (context: string, message: string, codeOrError: StatusCode | ApiError<any>, err?: any) => void;
  info: (context: string, message: string) => void;
  dml: IDmlMethods;
  loader: (context: string, message: string, code: StatusCode, count: number) => void;
  admin: <T>(context: string, callback: () => T | Promise<T>) => Promise<T | undefined>;
};

const DEFAULT_LOGGING: ILogFlags = {
  select: false,
  update: false,
  insert: false,
  delete: false,
  loader: true,
  account: false,
  errors: false,
  success: false,
};

/**
 * The core logging engine.
 *
 * @example
 * ```typescript
 * // Standard usage
 * log.success("Auth", "User Verified", SC.OK);
 *
 * // Atomic DML with metadata objects
 * log.dml.select("Orders", "SELECT * FROM...", ["id"], [123], { latency: '2ms' });
 *
 * // Security-wrapped admin logic
 * const result = await log.admin("DeleteUser", () => db.user.delete(id));
 * ```
 */
class Logger implements ILogger {
  /** @internal Environmental toggle flags parsed at startup. */
  private readonly flags: ILogFlags;

  constructor() {
    this.flags = this.getEnvFlags();
  }

  /** @internal Parses env flags once at startup. */
  private getEnvFlags(): ILogFlags {
    try {
      if (process.env.APP_LOGGING) {
        return { ...DEFAULT_LOGGING, ...JSON.parse(process.env.APP_LOGGING) };
      }
    } catch (error) {
      // Internal fallback to console for config errors
      console.error(`${Red("Error")} LogConfig: Malformed APP_LOGGING JSON.`, { error });
    }
    return DEFAULT_LOGGING;
  }
```

```
  /**
   * Late-binding accessor for the current {@link ISession}.
   * Ensures logs always reflect the most recent state of the 'Unified Passport',
   * even during phased boot sequences.
   *
   * @private
   * @readonly
   */
  private get _session(): ISession {
    return Session();
  }

  /**
   * Logs a successful operation to the console.
   *
   * @param context - The calling module or function name.
   * @param message - Optional custom message; defaults to the {@link StatusMessage}.
   * @param code - The {@link StatusCode} representing the result (Default: 200).
   */
  public success(context: string, message?: string, code: StatusCode = SC.OK): void {
    if (!this.flags.success || code >= 300) return;
    console.log(`${Green("Success")} ${Cyan(context)} [${this._session.alias}]: ${message || StatusMessage[code]}`);
  }

  public error(context: string, message: string, codeOrError: StatusCode | ApiError<any>, error?: any) {
    if (!this.flags.errors) return;

    // 1. Resolve the code early
    const incomingCode = codeOrError instanceof ApiError ? codeOrError.code : codeOrError;

    // 2. Silent Guard: If it's a numeric success/redirect code, skip logging as an "Error"
    if (typeof incomingCode === "number" && incomingCode <= 300) {
      return;
    }

    // 3. Normalization (Proceed only if it's a true error)
    const norm = codeOrError instanceof ApiError ? codeOrError : ApiError.normalize(context, error || codeOrError);
    console.error(`${Red("Error")} ${Cyan(context || error)} [${this._session.alias}]: ${message} (${norm.code}) ${norm.msg}`, norm.data || "");
  }

  public info(context: string, message: string) {
    console.log(`${Yellow("Info")} ${Cyan(context)} [${this._session.alias}]: ${message}`);
  }

  public loader(context: string, message: string, code: StatusCode = SC.OK, count = 0) {
    if (!this.flags.loader) return;

    if (count > 0 || code === 201) console.log(`${Green("Success")} ${Cyan(context)} [${this._session.alias}]: ${message || StatusMessage[code]}`);
    else console.log(`${Red("Failure")} ${Cyan(context)} [${this._session.alias}]: ${message || StatusMessage[code]}`);
  }

  /**
   * Enforces administrative authorization before executing logic.
   *
   * @template R - The expected return type of the callback.
   * @param context - Label for the security audit trail.
   * @param cb - The logic to execute if `isAdmin` is true.
   * @returns The result of the callback or undefined if authorization fails.
   * @throws {ApiError} 401 Unauthorized if the session is not an Admin.
   */
  public admin = async <R>(context = "Admin", cb: () => R | Promise<R>): Promise<R | undefined> => {
    const { user, alias, isAdmin } = this._session;

    if (!isAdmin) {
      const message = `${Magenta("Security")} ${Cyan(context)} [${alias}/${user}]: Unauthorized Admin Attempt`;
      console.warn(message);
      throw new ApiError(SC.UNAUTHORIZED_ACCESS, message, { ...this._session, context });
    }

    if (this.flags.info) {
      console.info(`${Magenta("Security")} ${Cyan(context)} [${alias}/${user}]: Authorized`);
    }

    return await cb();
  };

  /**
   * Dynamically generated Data Manipulation Language (DML) methods.
   * Provides `select`, `update`, `insert`, and `delete` handlers.
   *
   * @description
   * Methods support **Atomic Printing**: passing multiple objects as `meta`
   * arguments ensures they are logged as a single event without stringification.
   */
  public dml: IDmlMethods = DML_METHODS.reduce((acc, key) => {
    acc[key] = (context, sql, cols, args, ...meta) => {
      if (this.flags[key as keyof ILogFlags]) {
        console.log(`${Blue("SQL")} ${Magenta(key.toUpperCase())} ${Cyan(context)} [${this._session.alias}]:`, { sql, cols, args }, ...meta);
      }
    };
    return acc;
  }, {} as IDmlMethods);
}

/**
 * Singleton instance of the Logger.
 * Exported for global application use.
 */
export const log = new Logger();
```